Software Architecture (SCS 2303)
Assignment 3 – NexusEnroll: A University Course Enrolment System Modernisation
Software Design Analysis Report

Team Members:
[Name 1 – Registration No.]
[Name 2 – Registration No.]
[Name 3 – Registration No.]
[Name 4 – Registration No.]
[Name 5 – Registration No.]
[Name 6 – Registration No.]

University of Colombo School of Computing
August 2026

Contents

1 Introduction

Nexus University's 20-year-old LegacyEnroll system is monolithic, slow, and hard to extend. This report presents the architectural and detailed design for NexusEnroll, its replacement, and demonstrates that design with a runnable proof-of-concept in C++ (Crow framework) covering the Student, Faculty, and Administrator modules, alongside a modern Vanilla JavaScript Single Page Application (SPA).

Section 2 (Part A) selects and justifies an architectural pattern for the system as a whole. Section 3 (Part B) drills into the core business logic layer: the object model, the applied design patterns, UML views, and the software design principles the design adheres to. Section 4 maps the deliverable back to the source code accompanying this report.

2 Part A: Architectural Design

2.1 Chosen Architectural Pattern: Client-Server with CQRS Monolithic Backend

NexusEnroll adopts a modern Client-Server Architecture. The frontend is a decoupled Single Page Application (SPA) communicating over REST/JSON. The backend is a Modular Monolith built in C++ utilizing the CQRS (Command Query Responsibility Segregation) pattern to strictly separate read operations from write operations.

2.2 Justification

- **Performance and Speed:** Utilizing C++17 with the Crow microframework provides blazing-fast execution speeds, directly satisfying the requirement to handle high volumes of simultaneous users during peak enrolment windows without the overhead of massive microservice deployments.
- **Maintainability via CQRS:** LegacyEnroll's core problem is entangled codebase. By strictly separating Commands (mutations like enrolling in a course) from Queries (reads like viewing a catalogue), the business logic is decoupled. Each operation has its own isolated class, making the system incredibly easy to test and maintain.
- **Future Integration:** The API-first approach means that future systems (like a financial-aid system or mobile app) can seamlessly consume the exact same REST endpoints currently used by the web SPA, satisfying the 'same back-end services usable by both' requirement.
- **Data Integrity:** The use of explicit Repository patterns and transaction boundaries in C++ ensures that concurrent enrolments do not result in oversold seats.

2.3 Component Responsibilities

Component	Responsibility
Web SPA	Presentation tier. Consumes REST/JSON APIs only; written in Vanilla JS/HTML/CSS.
Crow API Router	Entry point for HTTP requests. Parses JSON and delegates to Commands/Queries.
CQRS Commands	Encapsulates business rules for data mutation (e.g., `EnrollStudentCommand`, `SubmitGradesCommand`).
CQRS Queries	Optimized read operations for the UI (e.g., `GetCourseCatalogueQuery`).
Repositories	Abstract data persistence (`CourseStore`, `UserStore`), interacting directly with MySQL.

3 Part B: Detailed Design & Implementation

3.1 Core Features In Scope

Per the assignment brief, the business tier covers three modules, matching the accompanying source code package:

- **Student:** browse catalogue, enrol/drop with full validation, view schedule, track waitlists.
- **Faculty:** view roster, draft and finalize grade submissions, submit capacity change requests.
- **Administrator:** manage courses/accounts, approve/reject faculty requests, generate enrolment/workload reports.

3.2 Design Patterns Applied

Pattern	Category	Where / Why
CQRS	Architectural	Strictly separates read operations (Queries) from writes (Commands). Solves complex data locking during peak enrolment by keeping reads lock-free and fast.
Repository	Structural	`CourseStore` and `UserStore` abstract MySQL queries. Allows the business logic to remain agnostic of the database schema.
Dependency Injection	Creational	The API routes receive a `Dependencies` struct at runtime containing initialized stores, allowing for easy unit testing via mocking.
Result Monad	Behavioural	A generic `Result<T>` wrapper is used throughout the C++ codebase to handle business errors (e.g., 'prerequisite missing') without using costly C++ exceptions.
Facade	Structural	The API routing layer (`routes.cpp`) acts as a Facade, hiding the complex instantiation and execution of CQRS commands from the frontend client.

3.3 UML Architectural Views

3.3.1 CQRS Architecture Class Diagram

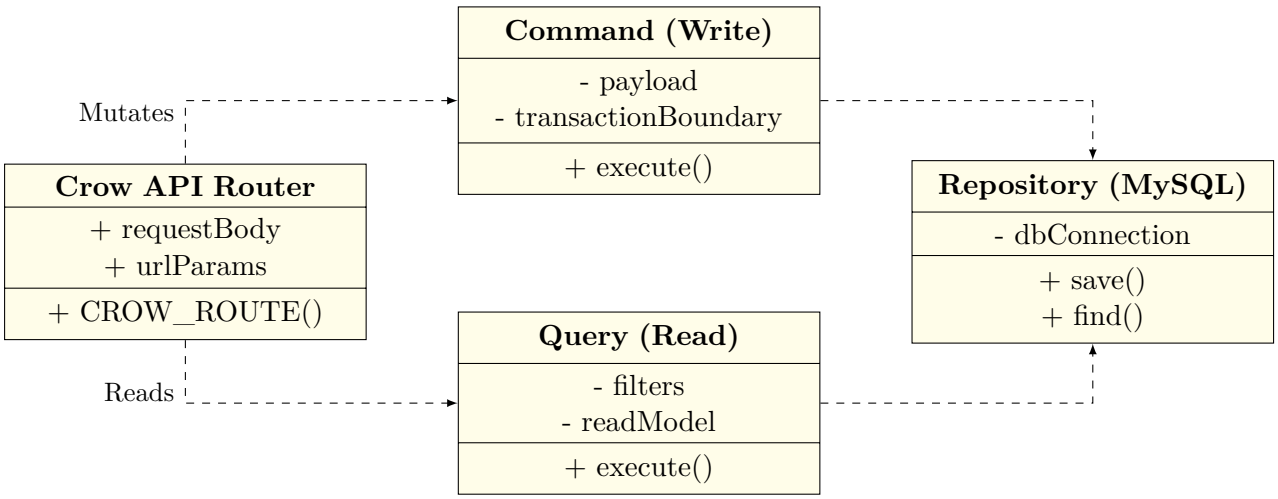

Figure 1: CQRS Architecture demonstrating separation of Commands and Queries.

3.3.2 Database Schema (ER Diagram)

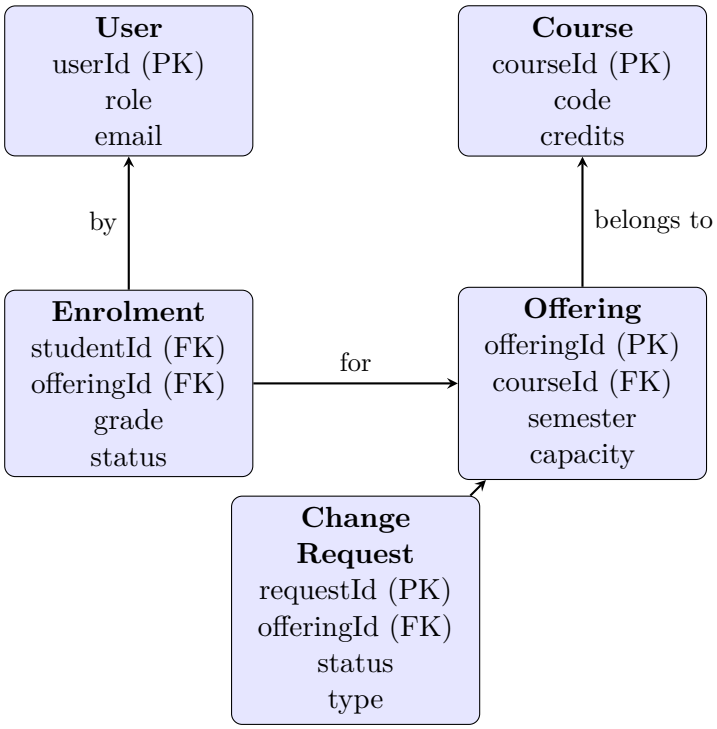

Figure 2: Simplified Entity-Relationship Diagram outlining the MySQL Database.

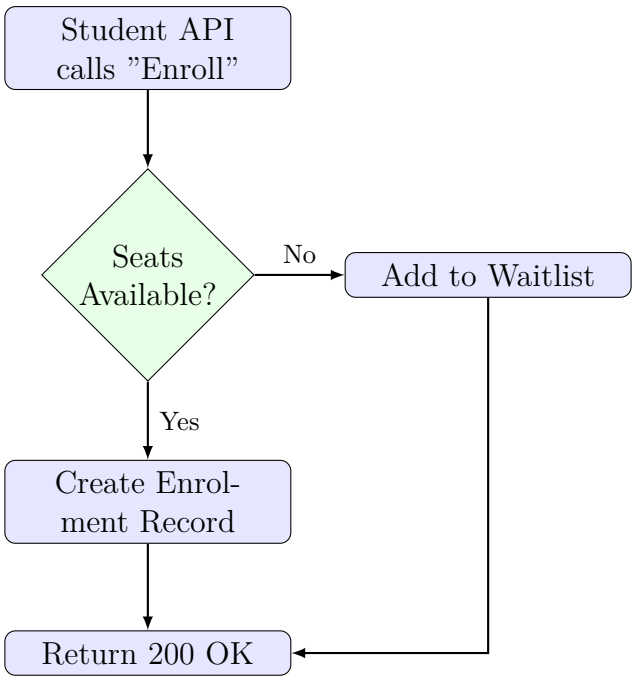

Figure 3: Course Enrolment Workflow Logic.

3.3.3 Course Enrolment Workflow

3.4 Software Design Principles

- **Single Responsibility:** A Command class like `EnrollStudentCommand` handles exactly one transaction. It delegates data fetching to the Repositories, adhering perfectly to SRP.
- **Open/Closed Principle:** New reports or features can be added by creating a new Query or Command class without modifying existing routing logic.
- **Dependency Inversion:** The API layer depends on abstract Command structures and Repository interfaces rather than concrete MySQL queries.
- **DRY:** Reusable utility functions for JSON parsing and error handling are centralized in `routes.cpp`.

3.5 Implementation Notes

The proof-of-concept is written in C++17 utilizing the Crow web framework and the MySQL C API. The frontend is a Vanilla HTML/CSS/JS Single Page Application served directly by Crow.

- **Backend Code:** Located in `src/`. Divided into `business/cqrs` (Commands/Queries), `data/mysql` (Repositories), and `presentation/api` (Routes).
- **Frontend Code:** Located in `frontend/`. Contains `index.html`, `style.css`, and `app.js`.
- **Database:** Managed via SQL scripts in `database/mysql/`.

How to run:

1. Initialize the MySQL database with `001_schema.sql` and `002_seed.sql`.
2. Compile the C++ backend using `make`.
3. Start the server via `make run`.
4. Navigate to `http://localhost:8080` and login as `U-STU-001` (Student), `U-FAC-001` (Faculty), or `U-ADM-001` (Admin).

3.6 Interactive Web Application (SPA UI Layer)

The team built a robust Single Page Application (SPA) utilizing HTML5, CSS3 Variables for dark-mode theming, and ES6 JavaScript to consume the C++ REST API seamlessly. The frontend routing is handled via hash-changes in `app.js`, dynamically rendering views based on user roles without page reloads. The UI successfully demonstrates live seat updates, dynamic waitlisting, and complex administrative reporting generated on-the-fly via the API.

4 Assessment Criteria Cross-Reference

Criterion	**Weight**	**Addressed In**
Architectural Choice	15%	Sections 2.1–2.2
Architectural Diagram	15%	Section 3.3.1
Design Pattern Application	40%	Section 3.2, source code
Implementation & Quality	20%	Section 3.5, source code
Documentation	10%	This report in full